

발표 연습용 대본 — 거울 앞 / 무대 직전 form

이 파일은 발표 직전 빠르게 훑어보고 거울 앞에서 연습할 수 있는 압축본. 백스테이지 메모와 의사결정 framing 은 학습용 ([dr-project-presentation-script-final.md](#)) 에 있고, 여기는 실제로 말할 멘트만 모아둔 form.

본인 파트 = 슬라이드 [6][7][40-46] 9 장 / 예상 시간 약 9 분

발표 직전 30초 체크 ★

- ☐ 톤 원칙 한 번 — 단순 한국어, 단언 회피, 졸업생 톤, 영업 멘트 X
- ☐ "본인" 빼기 — 제가 / 저희가 / 주어 생략
- ☐ 시계 한 번 — 본인 파트 9 분 안에 끝나는지
- ☐ 시연 영상 작동 확인 (백업본도)
- ☐ 물 한 잔
- ☐ 거울 한 번 — 건방진 톤 안 나오는지

슬라이드별 멘트 (말할 것만)

[6] Hybrid DR 아키텍처 — 시스템 소개 + 전체 구조 (~1분 30초)

"본격적인 아키텍처 설명에 들어가기 전에 짧게 — 저희 One-Sol 팀은 사내 재고관리 시스템에 Hybrid Cloud 기반의 DR 구조를 설계하는 것이 이번 프로젝트의 목표였습니다.

기존 구조는 사용자가 온프레임 HAProxy 를 거쳐서 WEB/WAS, 그리고 DB 까지 도달하는 단일 흐름 이었고, 온프레임 어딘가에 장애가 생기면 사용자가 재고관리 프로그램에 아예 접근할 수 없게 되어 업무 전체가 멈춰버리는 한계가 있었습니다.

그래서 저희가 선택한 게 *Pilot Light* 라는 DR 전략입니다. 평소에는 온프레임에서 정상 운영하면서, AWS 측에는 최소한의 자원만 대기시켜 두고, 장애가 발생하면 그 자원을 빠르게 깨워서 서비스 연속성을 보장하는 방식입니다. 비용은 낮게 유지하면서 연속성을 확보한다는 두 가지 장점 때문에 이 전략을 골랐습니다.

이 슬라이드가 그 Pilot Light 전략을 실제로 구성한 저희 시스템 전체 아키텍처입니다. 한눈에 복잡해 보이실 텐데, 큰 흐름 위주로 짚고 넘어가겠습니다.

왼쪽 사용자에서 시작합니다. HTTPS 요청이 들어오면 AWS Route53 → ALB 를 거치고, 평상시에는 ALB 가 AWS 측 HAProxy EC2 로 트래픽을 보내고, HAProxy 가 *Tailscale* 터널을 통해 오른쪽 온프레임 WEB/WAS 서버로 전달합니다. 즉 평소에는 AWS 를 거쳐서 결국 온프레임에서 처리되는 구조입니다.

AWS 측 SpringBoot ASG 는 평상시 0 대로 대기 하고 있다가, 재해가 발생하면 깨어나서 온프레임을 대신해 트래픽을 처리합니다. 이게 Pilot Light 의 깨어나는 자원 부분입니다.

DB 는 온프레임 MySQL → AWS DB EC2 → AWS RDS 로 *Cascade Replication* 을 통해 실시간 동기화됩니다. 그래서 재해 시에도 데이터 정합성이 유지됩니다. 이 부분은 뒤 4 번 챕터에서 자세히 다룹니다.

왼쪽 ETC 박스의 GitHub, Terraform, Ansible, 모니터링/Grafana 는 각각 형상관리, 인프라 코드화, 자동화, 모니터링 역할로 뒤 챕터에서 등장합니다.

다음 슬라이드에서는 같은 구조를 *네트워크 토폴로지* 측면에서 한 번 더 보여드리겠습니다."

[7] Hybrid DR 아키텍처 — 네트워크 토폴로지 (~30-40초)

"이 슬라이드는 [6] 의 전체 아키텍처를 *네트워크 트래픽 흐름* 측면에서 한 번 더 보여주는 슬라이드입니다.

왼쪽 온프레임 (192.168.20.0/24) 에는 WEB WAS, HAProxy, DB Master 가 있고, 오른쪽 AWS VPC (10.0.0.0/16) 에는 SpringBoot ASG, Jenkins, HAProxy, DB-EC2, RDS 가 있습니다.

두 환경을 잇는 핵심이 가운데 — 온프레임 HAProxy 가 *Tailscale Subnet Router* 로 동작해서, AWS 측 노드들이 이 라우터를 통해 온프레임 자원에 도달합니다. 그리고 AWS 쪽에서 *Tailscale 이 켜져 있지 않은 인스턴스* (예: SpringBoot ASG, RDS) 들도, VPC Route Table 의 라우팅 규칙을 통해 HAProxy AWS 를 거치면 결국 온프레임에 닿을 수 있는 구조입니다.

여기서는 *왜 이런 구조인지* 보다 *지금 어떻게 트래픽이 흐르는지* 만 짚고 넘어가겠습니다. 핵심은 한 가지 — 온프레임에서 단 한 대만 Tailscale 에 들어가고, 다른 노드들은 그 라우터를 거쳐서 AWS 와 통신한다는 점입니다. 이 결정의 이유는 뒤 CI/CD 챕터에서 한 번 더 자세히 다루겠습니다."

[40] Tailscale (~1분)

"Before 는 온프레임 6 대 모두에 Tailscale 을 설치하는 mesh 패턴, After 는 온프레임에서 HAProxy 만 Tailscale 을 설치하고 Subnet Router 로 두는 구조입니다. 호스트 수가 6 대에서 4 대로 줄어드는 효과가 있고, 이 비용 framing 은 마지막 *현재 한계와 보강 방향* 슬라이드에서 한 번 더 닫힙니다.

두 가지 결정이 들어갑니다.

첫째, *왜 Tailscale 이었는가* — Site-to-Site VPN 이나 Direct Connect 같은 옵션도 있었습니다. 그런데 Tailscale 은 *WireGuard 기반의 P2P VPN* 이라, 양쪽 끝에 별도 게이트웨이 장비를 두지 않아도 NAT 뒤에 있는 노드끼리 직접 연결됩니다. 공인 IP 가 따로 없어도 동작한다는 의미입니다. 그리고 무료 tier 가 있어서 국비 교육 프로젝트의 구축 환경에서 부담을 덜 수 있다는 점도 컸습니다.

둘째, *왜 HAProxy-only 패턴인가* — 처음에는 모든 노드에 Tailscale 을 설치하는 mesh 구조를 구상했는데, Tailscale 은 유료 plan 으로 넘어가면 *호스트 수가 비용을 결정하는* 구조입니다. 호스트 수를 줄이면 비용이 낮아지는 동시에 *보안 표면* 도 같이 줄어듭니다. 그래서 mesh 가 아닌 HAProxy-only 패턴을 채택했습니다.

Tailscale 은 저희 프로젝트가 Hybrid Cloud 로 성립할 수 있게 해주는 통로 자체입니다."

[41] 3 가지 시나리오 (~1분)

"이 슬라이드는 저희 시스템이 운영 중 마주할 수 있는 3 가지 시나리오를 한 화면에 정렬한 것입니다.

왼쪽 시나리오 1 은 *평시 앱 갱신* — 새 jar 가 출시됐을 때 ansible-playbook webwas.yml 한 번이면 끝나는 가장 단순한 흐름이고, 약 5 분 정도 예상했습니다.

가운데 시나리오 2 는 *Failover* — 온프레임에 장애가 발생했을 때 AWS 로 전환하는 흐름이고, 6 단계로 *RTO* 약 7-10 분 으로 예상했습니다.

오른쪽 시나리오 3 은 *Failback* — 온프레임이 복구된 뒤 다시 온프레임으로 돌아가는 흐름이고, 마찬가지로 6 단계 약 5 분 예상입니다.

두 가지 결정이 들어갑니다.

첫째, *시나리오 분기를 단일 Jenkinsfile 안에서 처리한 부분*. 시나리오마다 별도 파일을 만들 수도 있었지만, 안전장치 코드와 환경 변수, Tailscale 연결 정보가 시나리오 간에 겹치는 부분이 많았습니다. *단일 파일 + 분기 방식*이 코드 중복 회피와 일관성 측면에서 저희 환경에 더 적합하다고 봤습니다.

둘째, *시나리오를 정확히 셋으로 나눈 부분*. 운영자 입장에서 *결정이 필요한 순간이 이 셋*이라고 봤기 때문입니다. 크게 보면 — '새 배포가 나왔다', '장애가 발생했다 → AWS 로 옮기자', '복구됐다 → 온프레임으로 다시 돌아가자'. 운영자의 결정 단위 = 시나리오 단위, 이게 셋으로 나눈 출발점입니다."

[42] Jenkinsfile (~1분)

"이 슬라이드는 [41] 의 3 시나리오 분기가 *실제 Jenkinsfile 코드로 어떻게 구현되는지* 두 박스로 보여줍니다.

왼쪽 박스는 시나리오 분기 코드 — `parameters { choice() }` 한 블록입니다. 운영자가 Jenkins UI 에서 빌드를 시작할 때 'Phase 1 / 2 / 3' 중 하나를 드롭다운으로 선택하면, 그 선택이 빌드 내내 분기 기준으로 흘러갑니다.

오른쪽 박스는 시나리오 2 (Failover) 의 4 번째 stage — DB EC2 격리 SQL 입니다. `STOP SLAVE → RESET SLAVE ALL → super_read_only=ON`, 세 SQL 이 순서대로 실행됩니다.

두 가지 결정이 들어갑니다.

첫째, *왜 parameters { choice() } 였는가* — Jenkins 에는 webhook 트리거, Multibranch Pipeline, 별도 Job 분리 등 다른 트리거 방식도 있습니다. 그런데 *1-Click DR* 의 정의에서 가장 중요한 게 '사람 결정' 단계입니다. `choice()` 블록은 *운영자가 드롭다운에서 시나리오를 고르는 순간 = 사람 판단이 들어가는 단계*와 직접 매칭된다고 봤기 때문에, 다른 트리거 방식이 아니라 이 방식을 골랐습니다.

둘째, *DB EC2 격리 stage 가 왜 필요하고 SQL 이 왜 정확히 이 세 줄인가* — 사실 이 stage 를 빼고 단순 Failover 만 해도 빌드는 성공합니다. 하지만 *split-brain* 위험이 남아요. 온프레임 DB 가 살아 돌아왔을 때 EC2 측 DB 도 master 처럼 동작하면, *양쪽이 동시에 쓰기를 받아 데이터가 갈라집니다* — 같은 ID 에 다른 값이 들어가거나, 같은 키에 다른 사용자가 할당되는 식으로요. 합치려고 해도 어느 쪽이 진짜인지 알 수 없게 됩니다.

SQL 세 줄은 그걸 3 단계로 차단합니다 — `STOP SLAVE` 로 replication 중단, `RESET SLAVE ALL` 로 replication 메타정보 삭제, `super_read_only=ON` 으로 root 포함 모든 사용자의 write 차단. **한 줄이라도 빠지면 차단이 헐거워집니다.**

이 슬라이드는 *1-Click DR 이 단순 자동화가 아니라 안전장치가 박힌 자동화* 라는 걸 코드로 보여주는 자리입니다."

[43] Ansible 먹등성 (~1분)

"이 슬라이드는 시나리오 1 과 시나리오 3 의 Ansible 코드가 *얼마나 많은 부분을 공유하는지* 두 박스로 보여줍니다.

왼쪽 매트릭스는 어떤 role 이 어떤 호스트에 적용되는지 정리한 표입니다. *common* 과 *cloudwatch-agent* 는 모든 호스트, *springboot* 는 WEB/WAS, *mysql* 은 DB, *haproxy* 와 *tailscale* 은 HAProxy 호스트에만 적용됩니다.

오른쪽은 실제 코드 — 위쪽 박스가 시나리오 1 의 webwas.yml, 아래쪽 박스가 시나리오 3 의 site.yml 입니다. 위 박스의 common · springboot · cloudwatch-agent 세 role 이 아래 박스에서도 그대로 재사용됩니다.

두 가지 결정이 들어갑니다.

첫째, 왜 site.yml 과 webwas.yml 두 playbook 으로 분리했는가 — 시나리오마다 영향 범위가 다르기 때문입니다. 시나리오 1 은 새 jar 만 배포하니까 WEB/WAS 만 갱신하면 충분하고, 시나리오 3 은 온프레임 전체를 다시 프로비저닝해야 하니 모든 호스트가 대상입니다. 분리하니까 각 시나리오가 필요한 만큼만 실행됩니다.

둘째, 왜 같은 role 을 두 playbook 에서 재사용했는가 — 코드 중복을 피하기 위해서입니다. role 단위로 추출해서 양쪽이 같은 role 을 호출하면, 한 곳만 고쳐도 양쪽에 자동으로 반영됩니다. 이건 다음 [44] 슬라이드에서 보실 Terraform 모듈화에서도 똑같이 발현되는 장점입니다.

그리고 이 모든 게 멍등성 위에서 가능합니다. 멍등성은 같은 playbook 을 100 번 실행해도 100 번 모두 결과가 같은 성질입니다. Failover/Failback 빌드는 같은 코드가 시나리오마다 반복 실행되는데, 두 번 실행했을 때 결과가 달라지면 신뢰성이 무너집니다. Ansible 의 멍등성이 이걸 보장해주기 때문에, 같은 role 을 안전하게 반복 호출할 수 있습니다.

'단일 코드 + 환경별 분기' 원칙이 Jenkins 뿐 아니라 Ansible 에서도 같은 결로 들어갔다는 걸 보여주는 슬라이드입니다."

[44] Pipeline 이 Terraform 호출 (~1분)

"이 슬라이드는 [41] 의 3 시나리오에서 Pipeline 이 Terraform 을 어떻게 호출하는지 한 표로 정리한 것입니다.

시나리오 1 은 Terraform 호출이 없습니다 — ansible-playbook webwas.yml 한 번이면 끝입니다.

시나리오 2 는 terraform apply 인데, var-file 로 dr-active.tfvars 를 줍니다 — 결과적으로 ALB 가 SpringBoot TG 를 가리키고, ASG 가 0 → 2 로 늘어납니다.

시나리오 3 도 terraform apply 인데, var-file 만 pilot-light.tfvars 로 바꿉니다 — 결과는 반대로 ALB 가 HAProxy TG, ASG 가 2 → 0 으로 줄어듭니다.

시나리오 2 와 3 의 차이는 -var-file 한 줄 뿐입니다. 같은 main.tf, 다른 tfvars.

세 가지 결정이 들어갑니다.

첫째, 왜 같은 main.tf + 다른 tfvars 였는가 — 시나리오마다 별도 Terraform workspace 를 쓰거나, 별도 모듈 폴더로 분리하는 방식도 가능했습니다. 그런데 시나리오 2 와 3 의 인프라 차이가 사실은 몇 개 변수 값 차이 뿐이라서, 같은 main.tf 안에서 변수만 바꾸는 방식이 더 합리적이라고 봤습니다. 결과적으로 main.tf 가 단일 source of truth + tfvars 가 환경별 분리 라는 깔끔한 구조가 나왔습니다.

둘째, 왜 시나리오 1 에는 Terraform 호출이 없는가 — 시나리오 1 은 새 jar 만 배포하는 흐름이라 인프라 변경 이 0 입니다. 그런데도 terraform apply 를 호출하면 변경 사항 없음 이라는 plan 결과만 나오는데 plan 시건은 그대로 소비됩니다. 영향 범위에 맞춰서 도구를 호출하는 게 합리적이라고 봤습니다.

셋째, '정직한 기여' framing — Terraform 모듈 자체는 저희 팀과 AI 협업으로 작성했습니다. 제가 한 일은 tfvars 레이어 분리와 Pipeline 의 호출 흐름 통합 부분입니다.

Terraform 모듈 자체는 팀과 AI 협업의 결과물이고, 저는 거기서 tfvars 분리와 pipeline 통합 흐름을 맞췄습니다."

[45] 트러블슈팅 narrative (~1분 30초)

"이 슬라이드는 저희 프로젝트 구축 과정에서 마주했던 트러블슈팅 사례 중 세 가지를 도메인별로 정리한 것입니다 — Network, Cloud Security, Automation.

이 중에서 비중이 가장 큰 첫 번째 사례 — *Tailscale ProxyJump* 입니다.

처음에는 단순한 SSH 연결 문제로 보였습니다. ProxyJump 조합만 실패하는 상황이라, 4 시간을 디버깅에 쓰면서 '왜 이 조합만 안될까?' 라는 의문에서 벗어나지 못하고 있었습니다.

결국 AI 와 같이 다른 방안을 강구하던 과정에서 '*Tailscale Subnet Router* 로 우회하면 어떨까?' 라는 아이디어를 얻었고, 적용해보니 10 분 만에 해결했습니다.

결과적으로, 제 발표 첫 부분에 보여드렸던 *HAProxy-only Tailscale 아키텍처* 의 출발점이 되었습니다. 오류 해결 과정이 더 좋은 아키텍처로 이어진 사례입니다.

두 번째 *IAM Bootstrap* 이슈는 권한이 권한을 만들어야 하는, 닭과 달걀로 비유되는 문제였습니다. 사전에 Jenkins 인스턴스를 위해 분리해둔 admin 계정을 통해 비교적 간단하게 해결할 수 있었습니다.

세 번째 *Ansible: ansible_user* 문제는 Permission Denied 메시지에서 사용자명이 *jenkins* 로 잡히는 부분에서 의문이 출발했고, *ansible_user* 를 명시적으로 박는 한 줄로 해결했습니다.

세 사례가 도메인은 다 다르지만 흐름은 비슷했습니다. 의문은 제가 던지고, 진단의 가속은 AI 가, 결정은 다시 제가 했습니다. AI 가 답을 준 게 아니라, 답에 빨리 도달하게 도와주는 가속기에 가까웠어요."

[46] 현재 한계와 보강 방향 (~30초)

"마지막 슬라이드는 저희 시스템의 현재 한계와 다음 단계를 9 개 영역으로 정리한 것입니다. 우선순위에 따라 1순위 안정성 보강 / 2순위 운영 자동화 / 3순위 최적화, 세 컬럼으로 나뉩니다.

왜 1 / 2 / 3 순위로 나눴는가 — 9 개 영역을 그냥 나열하면 어디부터 손대야 할지 가늠하기 어렵습니다. 우선순위 기준은 사용자에게 직접적인 영향이 얼마나 가는가로 책정했습니다. 1 순위 안정성은 장애로 직결되는 영역, 2 순위 자동화는 운영 비용을 줄이는 영역, 3 순위 최적화는 있으면 좋은 영역으로 분류했습니다.

여기까지 구축했고, 여기부터는 다음에 풀 과제입니다."

Q&A 핵심 답변 (14 중)

Q1. Terraform 모듈은 본인이 직접 작성하셨나요?

"모듈 자체는 팀 + AI 협업으로 작성됐고, 제 기여는 tfvars 레이어와 Pipeline 통합입니다. 코드 라인 수보다 환경 전환 의사결정에 더 가까운 기여입니다."

Q2. Failover 빌드가 항상 성공하나요?

"오히려 lag check 안전장치 덕분에 replication 끊긴 상태에서는 의도적으로 실패합니다. 데이터 손실 방지가 최우선입니다."

Q3. 1-Click 인데 왜 인간이 결정해야 하나요?

"결정까지 자동화하면 일시적 장애에도 Failover 트리거되어 사용자 불편 + 비용 손해 + split-brain 위험이 생깁니다. 인간 판단으로 그걸 차단하는 게 운영 안정성 측면에서 더 나은 선택이라고 봤습니다."

Q4. RTO 가 왜 이렇게 긴가요? (3-4분대)

"안전장치 — replication lag check, DB 격리 SQL — 가 시간을 일부 소비합니다. 인스턴스 사이즈, 네트워크, 데이터량에 따라 더 짧아질 여지가 있습니다."

Q5. Failback 시간이 Failover 보다 긴 이유?

"Failback 은 의도된 계획 다운타임입니다. 안전한 원위치 복귀를 위해 정합성 검증 + 데이터 동기화에 시간을 씁니다. Failover 의 장애 복구 시간과는 의미가 다른 운영적 선택입니다."

Q6. 왜 Hybrid Cloud 였나요?

"기존 사내 시스템을 그대로 유지하면서 연속성을 추가로 확보하는 게 목적이었습니다. 온프레임을 모두 걷어내고 클라우드로 옮기는 것보다, 운영 환경은 그대로 두고 클라우드에 백업 환경을 두는 쪽이 비용과 운영 부담 면에서 합리적이라고 판단했습니다."

Q7. 왜 Jenkins 였나요?

"두 가지 이유입니다. Hybrid 환경에서 온프레임과 AWS 양쪽 자원에 모두 접근해야 했는데, Jenkins 는 자체 호스팅 방식이라 양쪽 네트워크에 직접 닿을 수 있다는 점이 맞았습니다. 그리고 `parameters { choice() }` 같은 운영자 입력 기반 빌드 트리거가 기본 기능으로 지원되어, 1-Click DR 의 *인간 결정* 단계에 자연스럽게 들어갔습니다."

Q8. 왜 Ansible 이었나요?

"역등성과 에이전트리스 구조 두 가지입니다. 역등성은 Failover/Failback 처럼 같은 코드를 시나리오마다 반복 실행하는 상황에서 신뢰성을 보장합니다. 에이전트리스는 대상 서버에 별도 프로그램 설치가 필요 없어서, 온프레임과 AWS EC2 양쪽을 같은 도구로 관리하는 부담이 적었습니다."

Q9. 왜 Tailscale 이었나요?

"WireGuard 기반 P2P 연결이라 별도 게이트웨이 장비 없이도 동작하고, 무료 tier 가 있어서 학습 환경에서 시작 부담이 적었습니다. 그리고 호스트 수가 곧 비용을 결정하는 구조라 운영 측면에서 비용 예측이 명확하다는 점도 맞았습니다."

Q10. Split-brain 이 뭔가요?

"단어 그대로 *분리된 뇌* — 하나의 시스템에 master 역할을 하는 노드가 둘 이상 동시에 존재하는 상황입니다. Failover 가 일어나서 AWS 측 RDS 가 master 가 됐는데, 그 사이 온프레임 DB 가 살아 돌아왔을 때 양쪽이 모두 master 처럼 쓰기를 받으면 데이터가 양쪽으로 갈라집니다. [42] 박스 ② 의 SQL 세 줄이 그걸 3 단계로 차단합니다."

Q11. 9 개 영역 중 가장 시급한 것 한 가지를 꼽는다면?

"1 순위 안에서도 HA (*Active-Standby + keepalived*) 입니다. HAProxy 가 단일 인스턴스라 SPOF (Single Point of Failure) 인데, 이게 다운되면 DR 전체 트리거 조건이 되어 비용·복구 시간이 오히려 늘어납니다. 가장 먼저 해결할 영역으로 봤습니다."

Q12. Active-Standby + keepalived 가 뭔가요?

"HAProxy 두 대를 띄워서 한 대는 Active 로 트래픽을 받고 한 대는 Standby 로 대기. *keepalived* 가 두 대 사이에서 *VIP (Virtual IP)* 를 관리하고, Active 가 죽으면 *VIP* 를 Standby 로 자동 이전해서 무중단 전환합니다. 인프라 도메인의 표준 HA 패턴입니다."

Q13. SLO + error budget 은 뭔가요?

"현재 모니터링이 *임계값 직관* — 예를 들어 *CPU 80% 넘으면 알람* — 으로 운영됩니다. 이걸 *Service Level Objective (SLO)* 기준으로 바꾸는 겁니다. '*한 달 안에 99.9% 가용성을 목표로 한다*' 같은 목표를 정하면, 거기서 깨질 수 있는 0.1% 가 *error budget*. 실제 사용자 영향 기준이라 알람 정확도가 올라갑니다."

Q14. Secret store 통합 또는 PITR 은 뭔가요?

"Secret store 는 인증 정보를 *AWS Secrets Manager* 같은 중앙 저장소로 통합 + 자동 회전 가능한 구조입니다. PITR 은 *Point-in-Time Recovery* 의 약자로, 백업 retention 안의 임의의 시점으로 RDS 복구할 수 있는 기능입니다. 둘 다 다음 단계로 학습하면서 적용할 영역입니다."

데이터 정합성 (Q&A 보완)

"데이터 정합성은 시연 영상에서 사용자 측면 (페이지 화면) 으로 확인했습니다. 추가로 cascade replication 통해 onprem master → onprem slave → AWS RDS 까지 실시간 동기화됨을 직접 SQL 로도 검증했습니다. 시간 관계상 시연 영상에는 사용자 측면만 담았습니다."

한 줄 요약 — 무대에 들고 들어갈 것

의사결정자 모드. 슬라이드는 청중이 알아서 읽고, 저는 *왜 이 결정을 했는지* 만 않는다.

의문은 제가, 진단의 가속은 AI 가, 결정은 다시 제가.